To enable HTTPS, HTTP/2, and HTTP/3, apply these modifications instead:

```
services:
  laravel.test:
    ports:
        - '${APP_PORT:-80}:80'
        - '${VITE_PORT:-5173}:${VITE_PORT:-5173}'
        - '443:443'
        - '443:443/udp'
    environment:
      SUPERVISOR_PHP_COMMAND: "/usr/bin/php -d variables_order=EGPCS
/var/www/html/artisan octane:start --host=localhost --port=443 --admin-
port=2019 --https"
      XDG_CONFIG_HOME:  /var/www/html/config
      XDG_DATA_HOME:  /var/www/html/data
```

**********

## # FrankenPHP via Docker

Using FrankenPHP's official Docker images can offer improved performance and the use of additional extensions not included with static installations of FrankenPHP. In addition, the official Docker images provide support for running FrankenPHP on platforms it doesn't natively support, such as Windows. FrankenPHP's official Docker images are suitable for both local development and production usage.

You may use the following Dockerfile as a starting point for containerizing your FrankenPHP powered Laravel application:

```
1   FROM dunglas/frankenphp
2
3   RUN install-php-extensions \
4       pcntl
5       # Add other PHP extensions here...
6
7   COPY . /app
8
9   ENTRYPOINT ["php", "artisan", "octane:frankenphp"]
```

**********

Then, during development, you may utilize the following Docker Compose file to run your application:

```yaml
# compose.yaml
services:
  frankenphp:
    build:
      context: .
    entrypoint: php artisan octane:frankenphp --workers=1 --max-requests=1
    ports:
      - "8000:8000"
    volumes:
      - .:/app
```

If the `--log-level` option is explicitly passed to the `php artisan octane:start` command, Octane will use FrankenPHP's native logger and, unless configured differently, will produce structured JSON logs.

**********

```
php artisan octane:start --server=frankenphp --
caddyfile=/path/to/your/Caddyfile
```

**********

## Swoole

If you plan to use the Swoole application server to serve your Laravel Octane application, you must install the Swoole PHP extension. Typically, this can be done via PECL:

```
pecl install swoole
```

### Open Swoole

If you want to use the Open Swoole application server to serve your Laravel Octane application, you must install the Open Swoole PHP extension. Typically, this can be done via PECL:

```
pecl install openswoole
```

Using Laravel Octane with Open Swoole grants the same functionality provided by Swoole, such as concurrent tasks, ticks, and intervals.

**********

### Swoole via Laravel Sail

> **!** Before serving an Octane application via Sail, ensure you have the latest version of Laravel Sail and execute `./vendor/bin/sail build --no-cache` within your application's root directory.

Alternatively, you may develop your Swoole based Octane application using Laravel Sail, the official Docker based development environment for Laravel. Laravel Sail includes the Swoole extension by default. However, you will still need to adjust the `docker-compose.yml` file used by Sail.

To get started, add a `SUPERVISOR_PHP_COMMAND` environment variable to the `laravel.test` service definition in your application's `docker-compose.yml` file. This environment variable will contain the command that Sail will use to serve your application using Octane instead of the PHP development server:

```
services:
  laravel.test:
    environment:
      SUPERVISOR_PHP_COMMAND: "/usr/bin/php -d variables_order=EGPCS
/var/www/html/artisan octane:start --server=swoole --host=0.0.0.0 --
port='${APP_PORT:-80}'"
```

**********

#### Swoole Configuration

Swoole supports a few additional configuration options that you may add to your `octane` configuration file if necessary. Because they rarely need to be modified, these options are not included in the default configuration file:

```php
'swoole' => [
    'options' => [
        'log_file' => storage_path('logs/swoole_http.log'),
        'package_max_length' => 10 * 1024 * 1024,
    ],
],
```

**********

Here we have different results when running the application using: **octane:start** OR **artisan serve**

```
$myStartTime = microtime(true);
Route::get('/', function () use($myStartTime) {
    $myLocalStartTime = microtime(true);
    return DateTime::createFromFormat('U.u', $myStartTime)->format("r (u)") .
    " - " .
    DateTime::createFromFormat('U.u', $myLocalStartTime)->format("r (u)");
});
```

**********

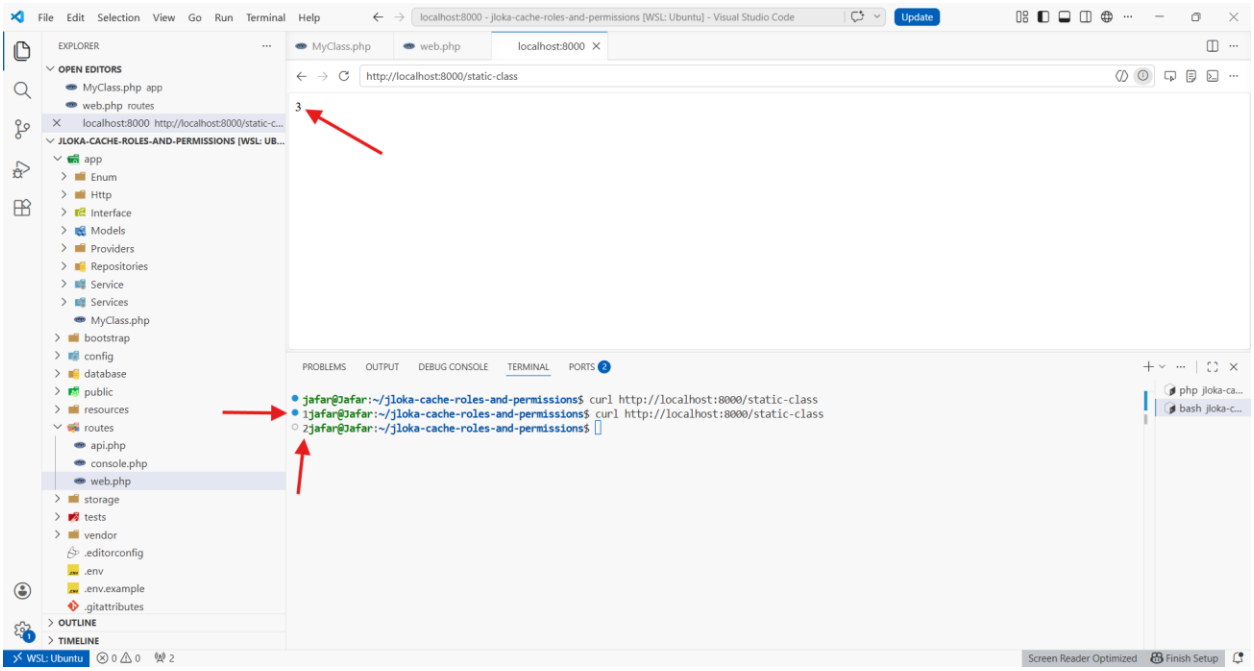
```
<?php

namespace App;

class MyClass
{
    public static $number = 0;
    /**
     * Create a new class instance.
     */
    public function __construct()
    {
         print "Construct\n";
    }

     public function __destruct()
    {
        print "Deconstruct\n";
    }

    public function add()
    {
        self::$number++;
    }

    public function get()
    {
        return self::$number;
    }
}
```

==========

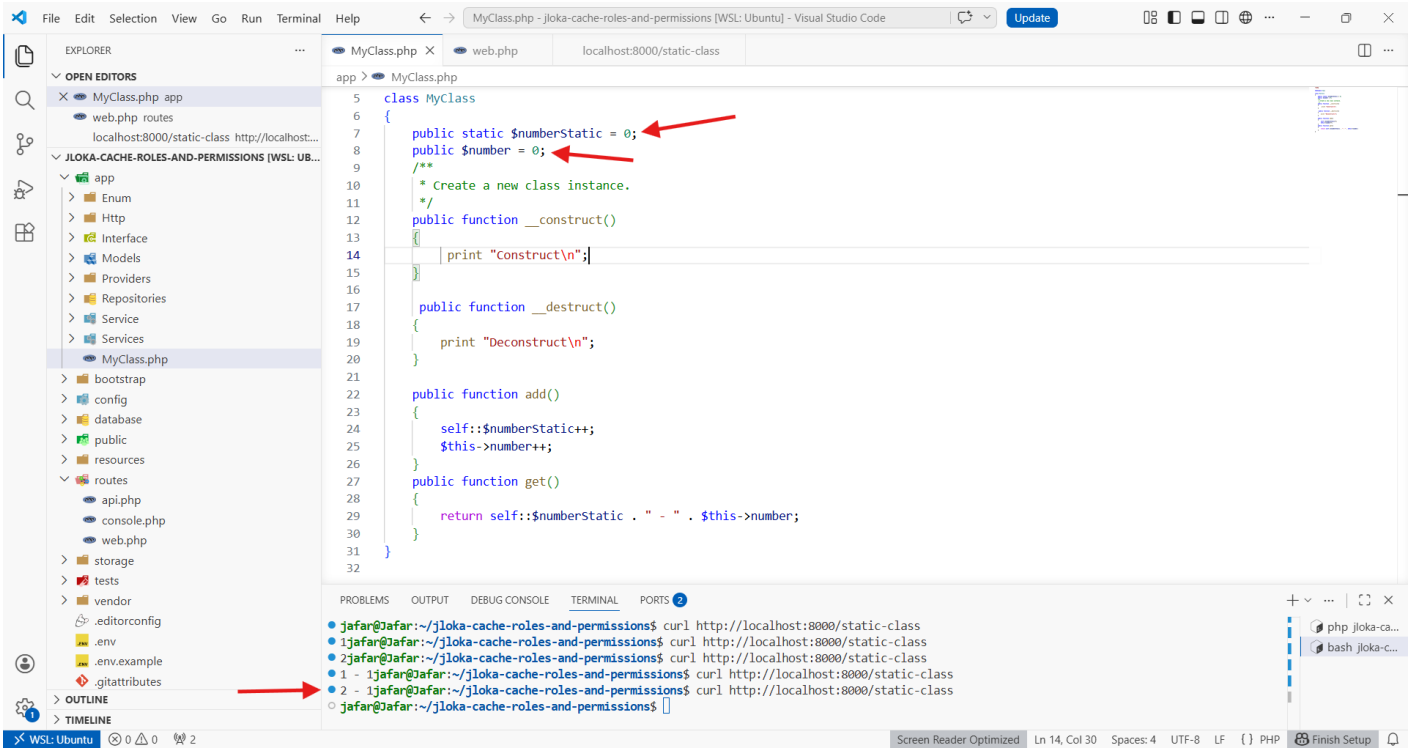
```php
use App\MyClass;
use Illuminate\Support\Facades\Route;

Route::get('/static-class', function (MyClass $myclass) {
    $myclass->add();
    return $myclass->get();
});
```

==========

When we run the application using: **php artisan octane:start**, the class with a static member will be instantiated once.




==========




**********

The lifecycle of a service provider starts with three phases: **creation**, **registration**, and **boot**.

**********

```php
<?php

namespace App\Providers;

use Illuminate\Support\ServiceProvider;

class MyServiceProvider extends ServiceProvider
{
     public function __construct($app)
    {
        echo "NEW      - " . __METHOD__ . PHP_EOL;
        parent::__construct($app);
    }

    public function register()
    {
        echo "REGISTER      - " . __METHOD__ . PHP_EOL;
    }
```

```
    public function boot()
    {
        echo "BOOT     - " . __METHOD__ . PHP_EOL;
    }
}
```

**********

The **register()** and **boot()** methods are needed **for dependency resolution**. First of all, all service providers are registered. Once they are all registered, they could be booted. **If a service provider** needs another service in the boot method, you can be sure that it is ready to be used because it is already registered.

**********

• **workers**: The number of workers that should be available to handle requests. We are going to set this number to 2.

• **max-requests**: The number of requests to process before reloading the server. We are going to set this number to a maximum limit of 5 for each worker.

**Example**: **php artisan octane:start** --**workers**=**2** --**max-requests**=**5**

**********

To use Roadrunner with Laravel: **composer require spiral/roadrunner-http nyholm/psr7**
composer require nyholm/psr7

Then we run: ./**vendor**/**bin**/**rr get-binary**

**********

A minimal configuration file requires a few things:

• The command to launch for each worker instance (**server.command**)

• The address and port to bind and listen for new HTTP connections (**http.address**)

• The number of workers to launch (**http.pool.num_workers**)

• The level of the log (**logs.level**)

**********

Simple RoadRunner Server example:

```php
<?php
require __DIR__ . '/vendor/autoload.php';

use Nyholm\Psr7\Response;
use Nyholm\Psr7\Factory\Psr17Factory;
use Spiral\RoadRunner\Worker;
use Spiral\RoadRunner\Http\PSR7Worker;

// Create new RoadRunner worker from global environment
$worker = Worker::create();

// Create common PSR-17 HTTP factory
$factory = new Psr17Factory();

$psr7 = new PSR7Worker($worker, $factory, $factory, $factory);

// creating a unique identifier specific to the worker
$id = uniqid('', true);
echo "STARTING $id";

while (true) {
    try {
        $request = $psr7->waitRequest();
        if ($request === null) {
            break;
        }
    } catch (\Throwable $e) {
        $psr7->respond(new Response(400));
        continue;
    }
```

```
    try {

        $psr7->respond(new Response(200, [], "Hello RoadRunner, Hello $id"));
        echo "RESPONSE SENT from $id";
    } catch (\Throwable $e) {
        $psr7->respond(new Response(500, [], 'Something Went Wrong!'));
    }
}
```

**********